

线性链表的链式存储结构

- **线性链表/单链表**: 链表中的每个结点只包含一个指针域。

- 数据元素之间的逻辑关系是由结点中的指针指示的。

- 单链表的存储结构:

```
typedef struct LNode{  
    ElemType data; // ElemType可以是任何C合法数据类型  
    struct LNode *next;  
} LNode, *LinkList;
```

- **头指针**: 用于指向表中第一个结点的LinkList型变量。

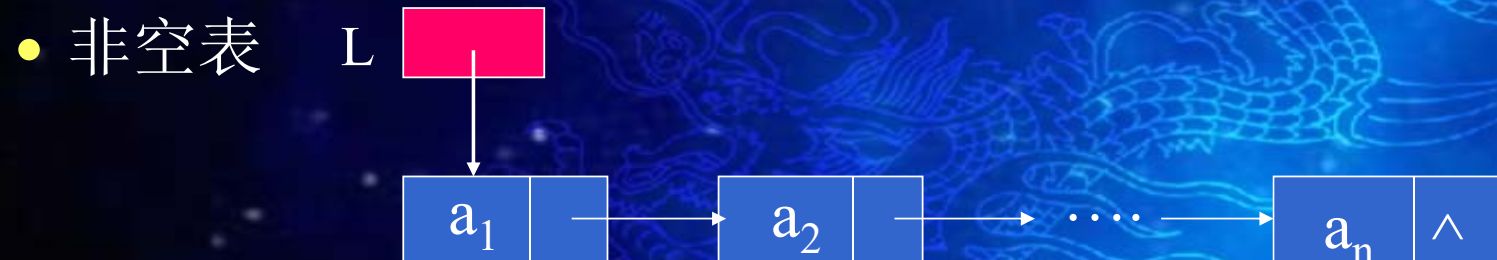
- 设L为LinkList型变量, 若L=NULL, 则为空表

- **头结点**: 在表的第一个结点之前附设一个结点, 其指针域指向第一个元素结点, 其数据域可以不存储任何信息, 也可用于存储表长等附加信息。

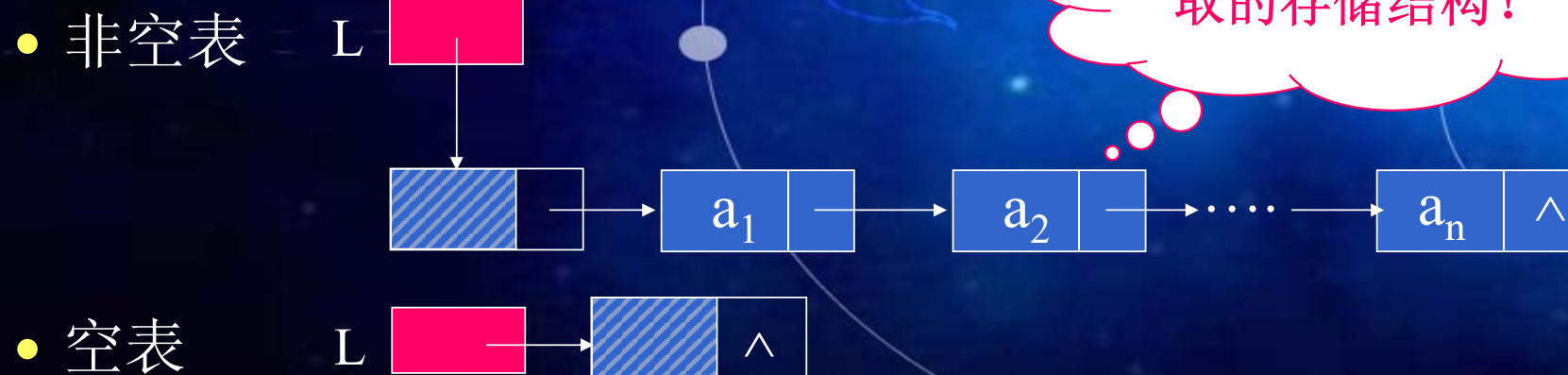
- 空表的头结点的指针域为“空”。

线性链表示意图

■ 不带头结点的单链表



■ 带头结点的单链表



单链表是非随机存取
的存储结构！

带头结点的单链表基本操作的实现

- 算法2.8 在带头结点单链表中求下标为I的元素
Status GetElem_L(LinkList L, int i, ElemType *e)
- 算法2.9 带头结点单链表的插入
Status ListInsert_L(LinkList L, int i, ElemType e)
- 算法2.10 带头结点单链表的删除
Status ListDelete_L(LinkList L, int i, ElemType *e)

算法2.8 GetElem

时间复杂度为 $O(n)$,
顺序表仅为 $O(1)$

```
Status GetElem_L(LinkList L, int i, ElemType *e) {  
    // L是带头结点的链表的头指针  
    // 当第i个元素存在时, 其值赋给*e并返回OK, 否则返回ERROR  
    p = L->next; j = 1;           // 初始化, p指向第一个结点, j为计数器  
    while (p && j < i) {          // 顺指针向后查找, 直到 p 指向第 i 个元素或p为空  
        p = p->next; ++j;  
    }  
    if ( !p || j > i )  
        return ERROR;           // 第 i 个元素不存在  
    *e = p->data;                // 取第 i 个元素  
    return OK;  
} // GetElem_L
```

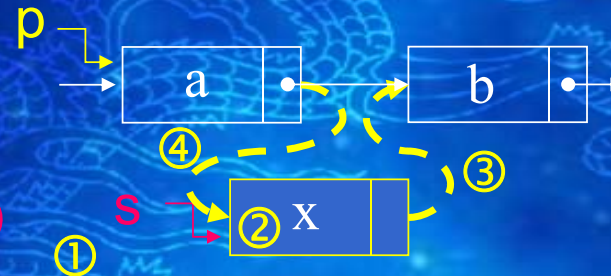

算法2.9 单链表的插入

■ 插入前



若单链表无头结点呢？

■ 插入后



区别何在？

```
Status ListInsert_L(LinkList L, int i, ElemType e) {
    //在带头结点的单链线性表L中第i个位置之前插入新元素 e
```

```
    p = L; j = 0;
```

```
    while (p && j < i-1)
```

```
        { p = p->next; ++j; }
```

```
    if (!p || j > i-1) return ERROR;
```

// 寻找第 i-1 个结点

// i 小于1或者大于表长

```
    ① s = (LinkList) malloc ( sizeof ( LNode)); // 生成新结点
```

```
    ② s->data = e;
```

```
    ③ s->next = p->next;
```

// 插入

```
    ④ p->next = s;
```

```
    return OK;
```

```
} // ListInsert_L
```

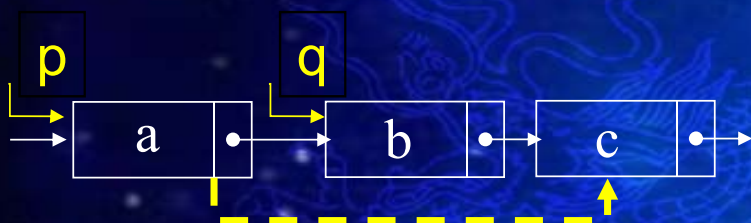
类似GetElem_L()

时间复杂度为O(n)

顺序可对换吗？



算法2.10 单链表的删除



若单链表无头结点呢?

```

Status ListDelete_L(LinkList L, int i, ElemType *e) {
    //在带头结点的单链线性表L中，删除第i个元素，并由*e返回其值
    p = L; j = 0;
    while ( p->next && j < i-1 ) { //寻找第 i 个结点，并令 p 指向其前趋
        p = p->next; ++j;
    }
    if ( !( p->next ) || j > i-1 )    // 删除位置不合理
        return ERROR;
    ① q = p->next; ② p->next = q->next; // 删除并释放结点
    ③ *e = q->data; ④ free(q);
    return OK;
} // ListDelete_L
    
```

类似GetElem_L()

时间复杂度为 $O(n)$

